Project Description: Rental Management Application

Project Overview:

A Rental Management Application is a comprehensive software solution developed to facilitate the management of rental properties, ensuring that property owners, property managers, and tenants can effectively and efficiently handle all aspects of the rental process. The application serves as a centralized platform for various tasks, including property listing, tenant onboarding, lease management, maintenance requests, and financial transactions.

Key Features and Functionalities:

1. Property Listings:
    - Property owners can create detailed listings of their rental properties, including property descriptions, rental rates, and high-quality photos.
    - Property managers can easily organize and promote available properties for rent.
2. Tenant Management:
    - Property managers can manage tenant information, including tenant applications, references, and lease agreements.
    - Tenants can submit applications, review lease terms, and communicate with property managers.
3. Lease Tracking:
    - The application helps property managers and property owners keep track of lease agreements, including lease terms, renewal dates, and rent payments.
4. Maintenance Requests:
    - Tenants can submit maintenance requests and track their status.
    - Property managers can assign and manage maintenance tasks efficiently.
5. Financial Management:
    - The application enables property owners and managers to handle financial transactions, including rent collection.
6. Communication and Messaging:
    - The application includes communication tools for property managers, property owners, and tenants to exchange messages and notifications.
7. Document Management:
    - Users can store and manage essential documents such as lease agreements, rental applications, and property inspection reports.
8. Reporting and Analytics:
    - The application provides insights into property performance, financial data, occupancy rates, and maintenance history through reports and analytics.
9. Security and Data Protection:
    - Robust security measures ensure the safety of sensitive data, including personal and financial information.

Group Project week 3

I. Topic-4_Reliable-Engineering (80%)

1. Make sure all Week 2 accepted work is already on main. Save or commit local changes before switching branches.
2. Download remote changes and update your local main.
3. Create a NEW branch from updated main for your assigned class. Do not work directly on main.
4. Open a pull request into main.
5. Repository owner merges approved pull requests into main. After each merge, confirm that the whole project still runs and all tests pass.
6. Update your Kanban board.

II. Make sure that your board mirrors your current progress. (20%)

Appendix

Validation requirements by class

Apply regex only where input has a meaningful text pattern. Apply numeric validation to numbers; use date objects/comparisons for dates. Keep inherited User validation in User instead of duplicating it in child classes.

Responsible class	Required validation	Example inputs / expected behavior
User	UserID: positive integer. FirstName and LastName: nonempty after trimming; allow ordinary names with letters, spaces, hyphens and apostrophes. Email: basic email-format regex. Validate constructor and UpdateProfile inputs.	Valid: UserID=2, "Anna", "Nguyen", anna@example.com. Invalid: UserID=-1; blank name; anna@@example.com. Invalid updates leave prior profile unchanged.
PropertyOwner	Inherit User checks through the parent constructor. For CreateProperty, delegate propertyID, description and rentalRate checks to the Property constructor; append only after successful creation.	Valid: propertyID=101, rate=5000. Invalid: rate=0; no Property is appended.
Property	PropertyID: positive integer. Description: nonempty after trimming (no restrictive regex needed). RentalRate: finite, positive decimal number; reject strings, booleans and non-finite values. Validate constructor and UpdateRentalRate.	Valid: 101, "Apartment A101", 5000.50. Invalid: ID=0, description=" ", rate=-50, rate="5000". On failed update, old rate remains.
Tenant	Inherit User checks. For RentProperty, rely on RentalContract validation; append to RentalContracts only after successful construction.	Invalid: unavailable property or reversed dates. No contract is appended when creation fails.
RentalContract	ContractID: positive integer. startDate/endDate: actual date values with endDate > startDate. Require available Property and appropriate Property/Tenant objects. Preserve existing state on failure.	Valid: 2026-11-01 to 2027-10-31. Invalid: endDate <= startDate, ID=-4, or already rented property. Invalid creation must not change IsAvailable.

Do not use regex to validate every field. Choose the validation technique that matches the data type.